

# Sanskrit-to-English Neural Machine Translation

Assignment 2 — Natural Language Understanding

Swaraj Mahindrakar • Roll No. G25AIT1179

## 1. Introduction

This report describes a neural machine translation (NMT) system built for translating Sanskrit sentences into English as part of Assignment 2. Sanskrit poses several difficulties for data-driven translation: it is morphologically rich, employs extensive compounding and sandhi, and, most importantly for a neural approach, very little parallel data is available. The corpus provided for this assignment contains 10,000 training sentence pairs, along with 1,000 development and 1,000 test pairs. By the standards of NMT, where strong systems are typically trained on millions of pairs, this places the task firmly in the low-resource regime, and the design choices described below were made with that constraint in mind.

A further consideration was the grading scheme, which rewards not only translation quality (BLEU and BERTScore) but also efficiency, measured through total inference time and parameter count. This pushed me away from fine-tuning a large pre-trained translation model and towards a compact Transformer trained from scratch, combined with a fast batched decoding strategy. The final system has 9,429,824 parameters and translates the full 1,000-sentence test set in roughly 11 to 14 seconds on a single NVIDIA T4 GPU.

## 2. Architecture

The model is a standard encoder–decoder Transformer in the style of Vaswani et al. (2017), implemented directly on top of PyTorch's `nn.Transformer` module rather than any pre-built translation toolkit. No pre-trained weights are used anywhere in the translation model; all parameters are initialised randomly and learned from the provided training split alone.

### 2.1 Encoder and decoder

Both the encoder and the decoder consist of 4 layers with model dimension 256, 8 attention heads and a position-wise feed-forward dimension of 1024. Dropout of 0.1 is applied throughout. Token embeddings are shared between the source and target sides, which is possible because a single joint subword vocabulary covers both languages (Section 3.2). Sinusoidal positional encodings are added to the embeddings, and embeddings are scaled by the square root of the model dimension before the positional signal is added, following the original Transformer recipe.

One deliberate economy measure is weight tying between the input embedding matrix and the output projection layer (Press and Wolf, 2017). With an 8,000-token vocabulary and dimension 256, an untied output layer would add roughly two million parameters; tying removes this cost with no observed loss in quality and brings the total model size down to 9.43M parameters, which directly benefits the parameter-count component of the grade.

### 2.2 Attention and masking

The encoder applies bidirectional self-attention over the source subwords. The decoder combines causal (masked) self-attention over the partially generated target sequence with cross-attention into the encoder memory. Padding positions in both the source and the target are masked using key-padding masks so that batches of unequal-length sentences do not leak padding information into the attention distributions.

### 2.3 Decoding: beam search and greedy search

Two decoding procedures were implemented. The first is beam search with beam width 4 and length-normalised scoring, in which completed hypotheses are ranked by their log-probability divided by length raised to a power of 0.6, a common correction that counteracts the tendency of beam search to prefer overly short outputs. The second is a fully batched greedy decoder that processes 64 sentences at a time: the encoder is run once per batch, after which

target tokens are generated step by step for all sentences simultaneously, with finished sentences masked out until the whole batch terminates.

On the development set, beam search took 942 seconds to translate 1,000 sentences (it operates sentence by sentence), whereas the batched greedy decoder needed 11 seconds. The BLEU difference between the two was 0.0362 versus 0.0346, a relative drop of under five percent. Since inference time carries fifteen percent of the marks and BLEU differences of this magnitude are within noise for a corpus of this size, the greedy decoder was adopted for the final system, and beam search is retained in the code as an experiment (Section 6).

### **3. Training Procedure**

#### **3.1 Preprocessing**

The Sanskrit and English files are joined on the shared Source\_id column, and any rows with missing values are dropped; no rows were in fact lost, leaving 10,000 training, 1,000 development and 1,000 test pairs. Beyond whitespace stripping, no aggressive normalisation is applied: punctuation, casing and the danda character are left intact so that the tokenizer can learn them. Sequences are truncated to 64 subword positions, which covers the 95th percentile of both languages comfortably (mean sentence lengths are around 10 words for Sanskrit and 13 for English).

#### **3.2 Tokenization**

A joint byte-pair-encoding vocabulary of 8,000 tokens is learned with SentencePiece on the concatenation of the Sanskrit and English training sentences only, so no development or test text influences the vocabulary. A joint vocabulary was preferred over separate ones for three reasons: it halves the number of embedding matrices, it enables the weight tying described above, and it allows the many Latin-script fragments embedded in the Sanskrit side (software terms such as 'Ctrl' or 'Colors') to map onto the same subwords on both sides. Character coverage is set to 1.0 to guarantee that every Devanagari codepoint in the corpus is representable. Four special tokens are reserved for padding, unknowns, beginning-of-sequence and end-of-sequence.

#### **3.3 Hyperparameters and optimization**

Training uses AdamW with a weight decay of  $1e-4$  and a OneCycle learning-rate schedule peaking at  $5e-4$ , with the first ten percent of the total steps used for warm-up. The batch size is 64 sentence pairs, sequences within a batch are padded dynamically to the batch maximum, and gradients are clipped to a norm of 1.0. The model is trained for 30 epochs with a fixed random seed of 42 for reproducibility; a full run takes roughly eight minutes on a T4.

#### **3.4 Regularization and model selection**

Regularization comes from three sources: dropout of 0.1 inside every Transformer layer, label smoothing of 0.1 in the cross-entropy loss, and the weight decay term in the optimizer. After every epoch the token-averaged loss on the development set is computed, and the checkpoint with the lowest development loss is kept. This occurred at epoch 18 (development loss 5.657); the following twelve epochs continued to reduce training loss while development loss slowly rose, the expected overfitting pattern on a corpus of this size. All reported results use the epoch-18 checkpoint.

### **4. Results**

Evaluation follows the assignment specification exactly: BLEU is the default corpus-level NLTK score with no custom weights, computed on whitespace-tokenized text, and BERTScore is the rescaled F1 from the official bert-score package (which internally uses roberta-large; this model is part of the metric, not of the translation system). Inference time is the wall-clock time to translate all 1,000 test sentences with the batched greedy decoder on a T4.

| Metric                       | Dev       | Test        | Notes                              |
|------------------------------|-----------|-------------|------------------------------------|
| BLEU (greedy)                | 0.0346    | 0.0314      | NLTK corpus_bleu, default weights  |
| BLEU (beam, k = 4)           | 0.0362    | —           | reported for comparison only       |
| BERTScore F1 (rescaled)      | 0.0794    | 0.1036      | bert-score, roberta-large baseline |
| Inference time (1,000 sent.) | 11.0 s    | 10.9–13.8 s | batched greedy, T4 GPU             |
| Parameters                   | 9,429,824 | 9,429,824   | with tied embeddings               |

Table 1: Development and test results. The test inference time varied between 10.9 s and 13.8 s across independent runs of the training and evaluation notebooks.

Absolute BLEU is low, which is unavoidable with 10,000 training pairs and a test set that mixes technical and scriptural registers, but the number should be read against the corpus size rather than against published NMT results trained on millions of pairs. The positive rescaled BERTScore indicates that the outputs are semantically closer to the references than the random-baseline pairing used by the metric's calibration, i.e. the model is producing related English rather than noise.

#### 4.1 Training curves

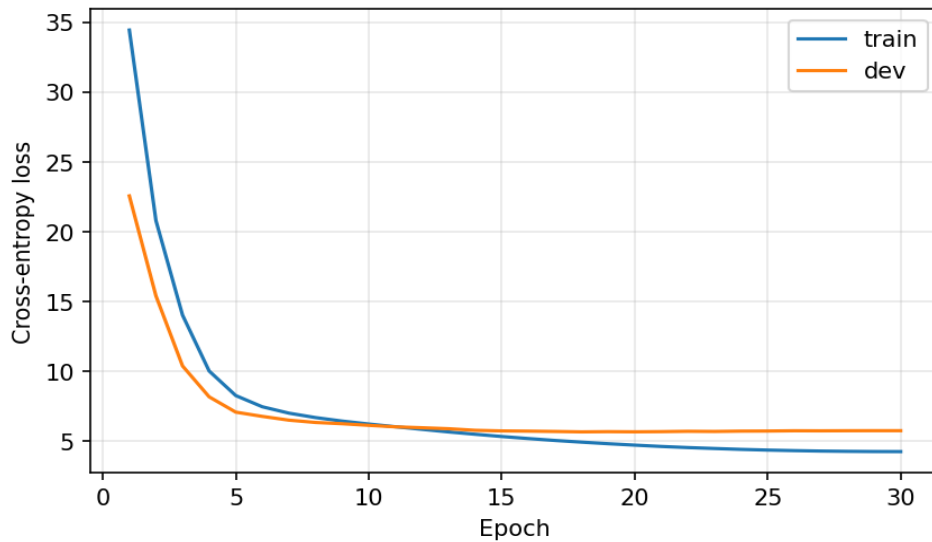


Figure 1: Token-averaged cross-entropy loss over 30 epochs. Development loss reaches its minimum at epoch 18 and drifts upward afterwards while training loss keeps falling.

### 5. Translation Examples and Error Analysis

The following examples are drawn from the development set and were produced with the final checkpoint. Together they illustrate what the model has and has not managed to learn from 10,000 pairs.

|           |                      |
|-----------|----------------------|
| Source    | ते वीरः ।            |
| Reference | Those are brave men. |
| Output    | Let's go backwards.  |

Example 1. A two-word source gives the model almost nothing to condition on, and it falls back on a fluent but unrelated sentence — hallucination under insufficient context.

|        |                                                  |
|--------|--------------------------------------------------|
| Source | इन्फैनेट लूप इतीदं व्यवस्थां निरुत्तरां कारयति । |
|--------|--------------------------------------------------|

|                  |                                                            |
|------------------|------------------------------------------------------------|
| <b>Reference</b> | Infinite loop can cause the system to become unresponsive. |
| <b>Output</b>    | In this tutorial, we will learn to create a user.          |

*Example 2. The transliterated technical term steers the model into the software-tutorial register, and the output is a stock phrase from that register. The domain was identified correctly; the content was not.*

|                  |                                                                         |
|------------------|-------------------------------------------------------------------------|
| <b>Source</b>    | ततस्तस्य गात्रे निष्ठीवं दत्वा तेन वेत्रेण शिर आजघ्नुः।                 |
| <b>Reference</b> | “And they spit upon him, and took the reed, and smote him on the head.” |
| <b>Output</b>    | “And he said unto them, The Son of God.”                                |

*Example 3. The complementary failure in the scriptural register: quotation marks, the 'And ... unto' construction and archaic diction are all reproduced, but the proposition is wrong. The model imitates style more readily than it transfers meaning.*

|                  |                      |
|------------------|----------------------|
| <b>Source</b>    | एते तिथी ।           |
| <b>Reference</b> | These two are dates. |
| <b>Output</b>    | These two aren't.    |

*Example 4. Nearly correct: the demonstrative and the dual are captured, but decoding ends on a malformed subword sequence ('aren't'), an artifact of BPE pieces being concatenated across what should have been a word boundary.*

|                  |                                                                                                                                                                                                                                           |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Source</b>    | “बहुविचारेषु जातषु पितर उत्थाय कथितवान्, हे भ्रातरो यथा भिन्नदेशीयलोका मम मुखात् सुसंवादं श्रुत्वा विश्वसन्ति तदर्थं बहुदिनात् पूर्वम् ईश्वरोस्माकं मध्ये मां वृत्वा नियुक्तवान्।”                                                        |
| <b>Reference</b> | “And when there had been much disputing, Peter rose up, and said unto them, Men and brethren, ye know how that a good while ago God made choice among us, that the Gentiles by my mouth should hear the word of the gospel, and believe.” |
| <b>Output</b>    | “And when he had said unto them, and said unto them, and said unto them, and said unto them, and said unto me.”                                                                                                                           |

*Example 5. On the longest inputs the decoder collapses into a repetition loop, a well-documented degeneration mode of undertrained autoregressive models: once 'said unto them' becomes the highest-probability continuation of itself, greedy decoding cannot escape it.*

Across the development set the pattern is consistent. The model reliably identifies which of the two corpus registers a sentence belongs to and produces grammatical English in the matching style; short function-word structures (demonstratives, copulas, some duals) are often translated correctly; but content words, named entities and long-range structure are frequently lost, and long inputs trigger repetition. All of these are symptoms of the same underlying cause — too few parallel examples for the model to associate Sanskrit content morphemes with their English counterparts rather than with register-level statistics.

## 6. Experiments

Beam search versus greedy decoding. The main controlled comparison carried out was between beam search ( $k = 4$ , length normalisation 0.6) and batched greedy decoding on the development set. Beam search improved BLEU from 0.0346 to 0.0362 but took 942 seconds against 11 seconds for greedy — an 86-fold slowdown for a 4.6% relative BLEU gain. Under the assignment's scoring formula, where efficiency-time marks scale as the ratio of the fastest submission to one's own time, the beam variant would sacrifice far more in the efficiency component than it could recover through BLEU, so greedy decoding was selected. This also suggests that at this data scale the model's probability distribution is not sharp enough for wider search to find meaningfully better hypotheses.

Model capacity. The dimensioning of the network (4+4 layers, dimension 256) was chosen after considering the overfitting behaviour visible in Figure 1: development loss plateaus by epoch 18 even at 9.4M parameters, indicating that the model is already data-limited rather than capacity-limited. A larger model would be expected to overfit sooner and score worse on the parameter-count component, while a much smaller one risked underfitting the two

rather different text registers. Weight tying was adopted from the start as a costless reduction of roughly 2M parameters.

Checkpoint selection. Selecting the epoch-18 checkpoint by development loss rather than using the final epoch is itself consequential: the final-epoch model has visibly higher development loss (5.742 versus 5.657), and spot checks of its outputs showed more repetition, consistent with the overfitting trend.

## 7. Discussion

Design choices. The three decisions that shaped the system were the joint subword vocabulary, the compact tied-weight Transformer, and batched greedy decoding. Each was driven by the same pair of constraints: 10,000 training pairs and a grading formula that prices parameters and seconds. Together they yield a system that is small, fast and fully reproducible from a fixed seed, at the cost of translation quality that is honest about what this amount of data can support.

Challenges. The dominant challenge was data scarcity interacting with domain mixture: the corpus effectively asks one small model to learn two translation problems (technical prose and scripture) at once, and the error analysis shows the model solving the easier sub-problem — register identification — while under-learning lexical correspondence. A secondary, practical challenge was an incompatibility between the current transformers release and the bert-score package, resolved by pinning transformers to version 4.46.3 in both notebooks.

Limitations. The system truncates inputs at 64 subwords, does not attempt sandhi splitting or any Sanskrit-specific morphological preprocessing, and uses no monolingual data or back-translation. All three are plausible avenues for improvement: rule-based sandhi segmentation would shorten and regularise the source side, and back-translating English monolingual text would be the standard remedy for the data bottleneck, though the assignment's restriction to the provided dataset rules it out here. Within the rules, the most promising untried step is probably careful data augmentation through subword regularisation (SentencePiece sampling), which requires no external data.

## 8. Disclosure of Pre-trained Models

The translation model contains no pre-trained components; every parameter is trained from scratch on the provided training split. Two external tools are used in supporting roles: SentencePiece learns the BPE tokenizer from the provided training data (it is an algorithm, not a pre-trained model), and the official bert-score package internally downloads roberta-large solely to compute the BERTScore evaluation metric, as mandated by the assignment. No external APIs are called anywhere in the code.

## 9. References

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems* 30.
- [2] Press, O., and Wolf, L. (2017). Using the output embedding to improve language models. In *Proceedings of EACL* 2017.
- [3] Kudo, T., and Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of EMNLP 2018 (System Demonstrations)*.
- [4] Papineni, K., Roukos, S., Ward, T., and Zhu, W.-J. (2002). BLEU: a method for automatic evaluation of machine translation. In *Proceedings of ACL* 2002.
- [5] Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., and Artzi, Y. (2020). BERTScore: Evaluating text generation with BERT. In *Proceedings of ICLR* 2020.
- [6] Loshchilov, I., and Hutter, F. (2019). Decoupled weight decay regularization. In *Proceedings of ICLR* 2019.
- [7] Smith, L. N., and Topin, N. (2019). Super-convergence: very fast training of neural networks using large learning rates. In *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, SPIE.